Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.:T039	Name:Bhavesh Ramakant Salaskar
Class:TYIT	Batch:1
Date of Assignment:	Date/Time of Submission:

Part B - Program

a) Create a web application to demonstrate the use of different types of Cookies.

Steps:

1. Create Project: Open Visual Studio > Create new ASP.NET Web Application (.NET Framework) > Select Web Forms.
2. Design WebForm1: Add TextBox1, Button1 (ViewState counter), Button2 (Save Cookie & Redirect), Label1, and Label2.
3. Write Code for WebForm1: In Page_Load, track and increment ViewState["count"]. In Button2_Click, create HttpCookie("name"), store value, and redirect.
4. Design & Code WebForm2: Add code in Page_Load to check for Request.Cookies["name"] and display greeting.
5. Run Project: Set WebForm1.aspx as Start Page and press F5 to test.

CSharp

```
using System;
using System.Web;
using System.Web.UI;
namespace Practical5c {
    public partial class WebForm1 : Page {
        protected void Page_Load(object sender, EventArgs e) {
            if(IsPostBack) {
                int ViewStateVal =
Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "view state =" +
ViewStateVal.ToString();
            } else { ViewState["count"] = 1; }
        }
        protected void Button2_Click(object sender, EventArgs e) {
```

```
            HttpCookie h = new HttpCookie("name");
            h.Value = TextBox1.Text;
            Response.Cookies.Add(h);
            Response.Redirect("Webform2.aspx");
        }
    }
}
```

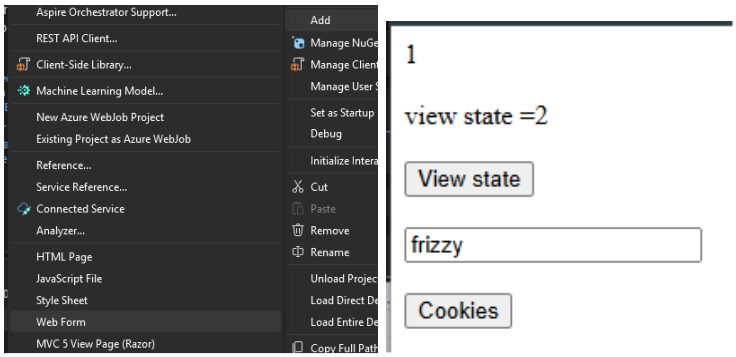
Webform2

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical5c
{
   public partial class WebForm2 : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
         if (Request.Cookies["name"] != null)
         {
            Response.Write("Welcome" + Request.Cookies["name"].Value);
         }
      }
   }
}
```

Output:

Welcomefrizzy

Label



b) **Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.**

Steps :-

1. Create Project: Open Visual Studio Create new ASP.NET Core Web App (MVC).
2. Install Packages: Open Package Manager Console and run the necessary Entity Framework Core commands.
3. Install-Package Microsoft.EntityFrameworkCore.SqlServer
4. Install-Package Microsoft.EntityFrameworkCore.Tools
5. Define Model & DbContext: Create Product.cs (Id, Name, Price) and AppDbContext.cs containing DbSet<Product> Products.
6. Configure Database Connection: Add connection string in appsettings.json and register AddDbContext in Program.cs.
7. Execute Insertion: Run Add-Migration & Update-Database in console. Add insertion logic inside HomeController.cs.

Code:

HomeController.cs:

```
using Microsoft.AspNetCore.Mvc;
using practical5b.Models;

public class HomeController : Controller
{
  private readonly AppDbContext _context;

  public HomeController(AppDbContext context)
  {
    _context = context;
  }

  public IActionResult Index()
  {
    Product p = new Product();

    p.Name = "Laptop";
    p.Price = 50000;

    _context.Products.Add(p);
    _context.SaveChanges();

    return Content("Product Inserted Successfully");
  }
}
```

AppDbContent.cs:

```
using Microsoft.EntityFrameworkCore;
namespace practical5b.Models
{
  public class AppDbContext:DbContext
```

```
  {
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {

    }
    public DbSet<Product> Products { get; set; }
  }
}
```

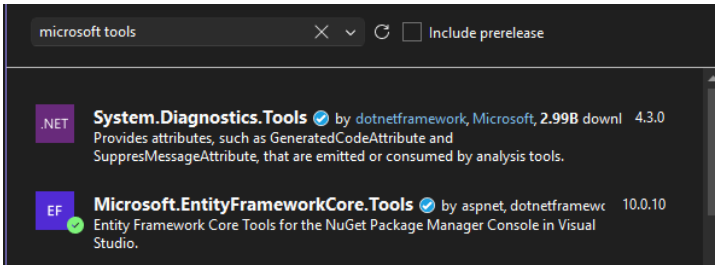
Product.cs:

```
namespace practical5b.Models
{
  public class Product
  {
    public int Id { get; set; }
    public string Name { get; set; }
    public int Price { get; set; }
  }
}
```

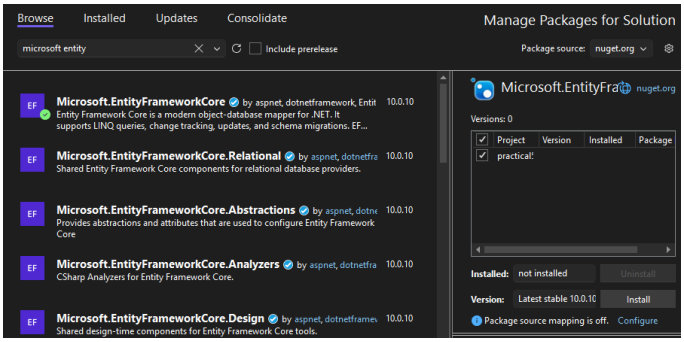
Program.cs:

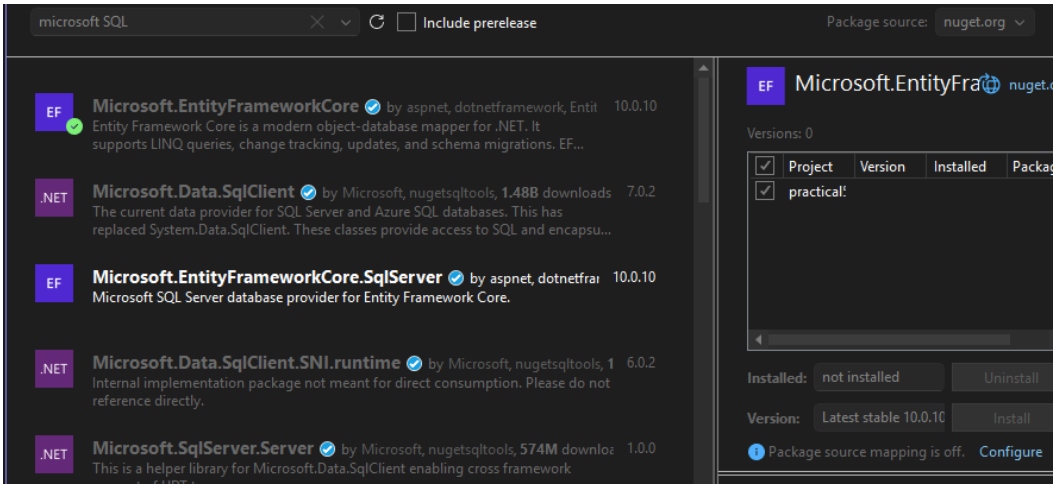
```
using Microsoft.EntityFrameworkCore;
using practical5b.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

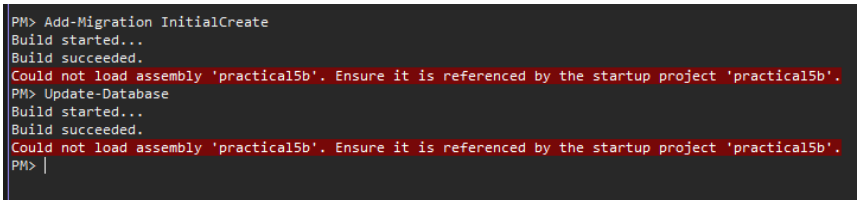
Output:




```
Product Inserted Successfully
```







C) Develop a web application to perform CRUD Operations on Employee table using EF Core.

Steps :-

1. Define Model: Create Employee.cs with properties (Id, Name, Department, Salary).
2. Setup DbContext: Create AppDbContext.cs with DbSet<Employee> Employees and register it inside Program.cs.
3. Apply Migrations: Run Add-Migration AddEmployeeTable followed by Update-Database in Package Manager Console.
4. Build Controller Actions: In EmployeesController.cs, implement methods for Create, Read, Update, and Delete operations.

5. Create Views: Generate corresponding Razor views under Views/Employees/ (Index, Create, Edit, Delete).

Code:

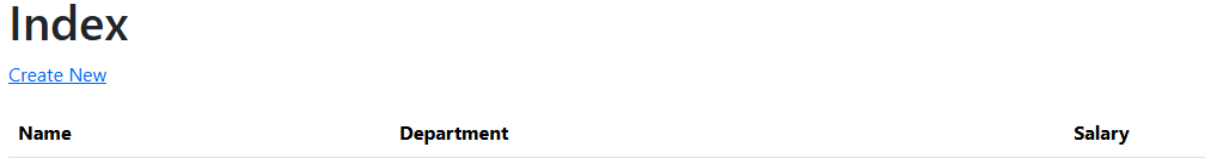
Employee.cs:

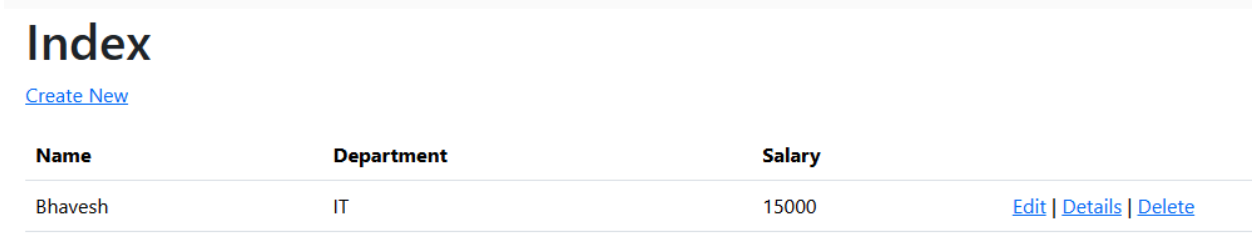
```
namespace PracticalC.Models
{
   public class Employee
   {
      public int Id { get; set; }
      public required string Name { get; set; }
      public required string Department {  get; set; }
      public Double Salary {  get; set; }


   }
}
```

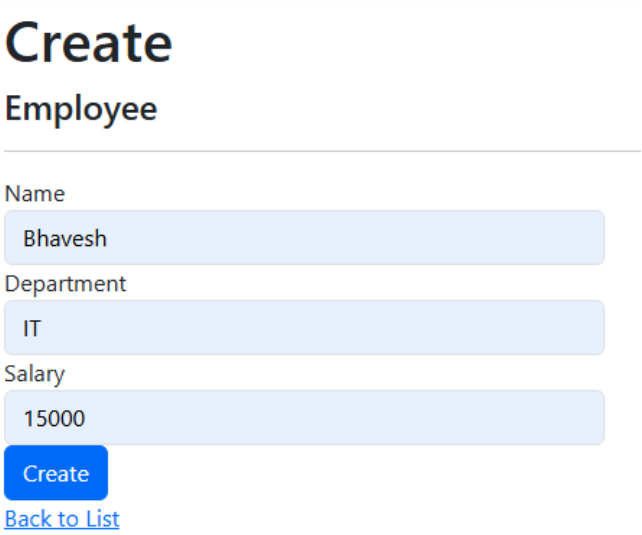
Appsettings.json:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServe
rCertificate=True",
    "AppDbContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleAc
tiveResultSets=true"
  }
}
```

Output:







d) Authentication and Authorization in ASP.NET Core.

Steps :-

1. Register Auth Services: In Program.cs, add services using AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(...) and AddAuthorization().
2. Configure Pipeline: Add app.UseAuthentication() before app.UseAuthorization() in Program.cs.
3. Build Login Logic: In HomeController.cs, validate user credentials, create a ClaimsIdentity, and call await HttpContext.SignInAsync(...).
4. Protect Route: Add the [Authorize] attribute directly above the Secure() action method.
5. Build View & Logout: Create Views/Home/Index.cshtml with a login form, and implement await HttpContext.SignOutAsync(...) inside the Logout() action.

Code:

Algorithm

8. Create an ASP.NET Core MVC project.
9. Enable Cookie Authentication in Program.cs.
10. Create Login action in HomeController.

11. Verify username and password.
12. Redirect to Secure page after successful login.
13. Use [Authorize] to protect the Secure page.
14. Logout and redirect to Login.

Program.cs

```
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
.AddCookie(options =>
{
   options.LoginPath = "/Home/Index";
});

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
```

```
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;

namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Index(string username, string password)
        {
            if (username == "admin" && password == "123")
            {
                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.Name, username)
                };
```

```
            var identity = new ClaimsIdentity(claims,
                CookieAuthenticationDefaults.AuthenticationScheme);

            await HttpContext.SignInAsync(
                CookieAuthenticationDefaults.AuthenticationScheme,
                new ClaimsPrincipal(identity));

            return RedirectToAction("Secure");
        }

        ViewBag.Message = "Invalid Username or Password";
        return View();
    }

    [Authorize]
    public IActionResult Secure()
    {
        return Content("Welcome! Login Successful.");
    }

    public async Task<IActionResult> Logout()
    {
        await HttpContext.SignOutAsync(
            CookieAuthenticationDefaults.AuthenticationScheme);

        return RedirectToAction("Index");
    }
  }
}
```

Views/Home/Index.cshtml

```
<h2>Login</h2>

<form method="post">

    Username :

    <input type="text" name="username" /><br /><br />

    Password :

    <input type="password" name="password" /><br /><br />

    <input type="submit" value="Login" />

</form>

<p style="color:red">

    @ViewBag.Message

</p>
```

Output: